TA145 — Redes Neuronales y Aprendizaje Profundo

Trabajo Práctico N°1: Redes de Hopfield

Manuel Hirsch

Padrón: 110221

mhirsch@fi.uba.ar

Índice

1. Introducción Teórica **3**
1.1. Arquitectura y Regla de Aprendizaje . . . 3
1.2. Dinámica de Recuperación y Energía . . . 3
1.3. Capacidad de Almacenamiento . . . 3
1.4. Estados Espurios . . . 3
1.5. Análisis Estadístico de la Capacidad y Probabilidad de Error . . . 3

2. Ejercicio 1 **4**
2.1. Consigna . . . 4
2.2. Resolución . . . 4
2.2.1. Preprocesamiento . . . 4
2.2.2. Verificación de entrenamiento . . . 4
2.2.3. Patrones alterados . . . 5
2.2.4. Estados espurios . . . 6
2.2.5. Entrenamiento de la red con los 6 patrones . . . 6

3. Ejercicio 2 **7**
3.1. Consigna . . . 7
3.2. Resolución . . . 7
3.2.1. Metodología experimental para patrones descorrelacionados . . . 7
3.3. Generación de patrones correlacionados . . . 8
3.3.1. Capacidad de la red en función de la correlación . . . 9

4. Ejercicio 3 **10**
4.1. Consigna 10
4.2. Resolución 10
4.2.1. Error frente a perdida de sinapsis 10
4.2.2. Capacidad frente a pérdida de sinapsis 10
4.2.3. Energía ante actualización asincrónica 12

1. Introducción Teórica

Las redes de Hopfield son un tipo de red neuronal recurrente que operan como sistemas de memoria asociativa. A diferencia de las memorias tradicionales que se acceden mediante direcciones explícitas, estas redes permiten recuperar un patrón completo almacenado a partir de una versión parcial, ruidosa o alterada del mismo.

1.1. Arquitectura y Regla de Aprendizaje

La red está compuesta por N neuronas interconectadas de manera completa y simétrica. Los estados de las neuronas $S_i \in \{-1, 1\}$ toman valores bipolares. Los pesos sinápticos w_{ij} entre la neurona i y la neurona j son simétricos ($w_{ij} = w_{ji}$) y se asume que no existen autoconexiones ($w_{ii} = 0$).

Para almacenar un conjunto de p patrones de entrenamiento ξ^μ (con $\mu = 1, \ldots, p$), se utiliza una generalización de la regla de Hebb:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu \quad \text{para } i \neq j \tag{1}$$

1.2. Dinámica de Recuperación y Energía

Dado un patrón inicial de entrada (que puede contener ruido o información faltante), la red evoluciona actualizando el estado de sus neuronas según la regla:

$$S_i = \text{sign}\left(\sum_{j \neq i} w_{ij} S_j \right) \tag{2}$$

Cada actualización asincrónica de una neurona disminuye o mantiene igual el valor de una función matemática que describe globalmente a la red, conocida como función de *energía* o función de Lyapunov. Este proceso iterativo continúa hasta que la red alcanza un estado estable (un mínimo local en el paisaje de energía), que idealmente corresponde a uno de los patrones originales memorizados.

1.3. Capacidad de Almacenamiento

A medida que se agregan más patrones a la red, la interferencia o *crosstalk* entre ellos aumenta. Si la cantidad de patrones p es demasiado grande en relación a la cantidad de neuronas N, los mínimos de energía se deforman. Como se demuestra en la literatura teórica (Hertz et al., 1991), para mantener una probabilidad de error baja, la capacidad máxima teórica de la red está dada por:

$$p_{max} \approx 0{,}138N \tag{3}$$

Superar este límite provoca un colapso en la memoria asociativa, donde la red pierde abruptamente su capacidad de recuperar los patrones de forma correcta.

1.4. Estados Espurios

La regla de Hebb garantiza la creación de mínimos de energía para los patrones deseados, pero como efecto secundario matemático, también genera mínimos locales adicionales no deseados. Si el estado de la red cae en uno de estos atractores, se dice que convergió a un *estado espurio*. Los más relevantes son:

- **Patrones inversos:** Por la simetría de la regla de aprendizaje, si ξ^μ es un atractor estable de la red, su versión invertida $-\xi^\mu$ también lo será.
- **Estados de mezcla (Mixture states):** Son estados estables que resultan de la combinación lineal de un número impar de patrones almacenados originalmente.

1.5. Análisis Estadístico de la Capacidad y Probabilidad de Error

Para comprender los límites de almacenamiento de la red de Hopfield, es necesario analizar el campo local h_i^ν que recibe una neurona i cuando se intenta recuperar un patrón específico ξ^ν. Reemplazando la regla de Hebb en la ecuación de actualización, el campo de entrada se puede dividir en dos términos:

$$h_i^\nu = \sum_{j \neq i} w_{ij} \xi_j^\nu = \xi_i^\nu + \frac{1}{N} \sum_{j \neq i} \sum_{\mu \neq \nu}^{p} \xi_i^\mu \xi_j^\mu \xi_j^\nu \tag{4}$$

El primer término (ξ_i^ν) es la *señal* deseada, que empuja a la neurona hacia el estado correcto. El segundo término es la interferencia o *crosstalk* generado por el resto de los patrones almacenados.

Si asumimos que los patrones almacenados son pseudo-aleatorios (es decir, estadísticamente independientes y con igual probabilidad de valer $+1$ o -1), el término de *crosstalk* se comporta como una variable aleatoria que sigue una distribución normal (Gaussiana) con media cero y varianza $\sigma^2 = p/N$.

Para que la red cometa un error en la neurona i, el término de *crosstalk* debe tener un signo opuesto a ξ_i^ν y una magnitud mayor a 1, superando así la señal deseada. La probabilidad teórica de que este error ocurra al intentar recuperar un bit se calcula integrando la cola de la distribución Gaussiana:

$$P_{error} = \frac{1}{\sqrt{2\pi}} \int_{\sqrt{N/p}}^{\infty} e^{-t^2/2} dt = \frac{1}{2} \left[1 - \operatorname{erf}\left(\sqrt{\frac{N}{2p}} \right) \right] \tag{5}$$

Esta ecuación demuestra que la probabilidad de error (P_{error}) depende exclusivamente de la relación entre la cantidad de patrones y el tamaño de la red, es decir, el cociente p/N (o P_{max}/N como se expresa en el Trabajo Práctico). A medida que se intenta almacenar una mayor cantidad de patrones en una red de tamaño fijo, la varianza de la interferencia aumenta y, en consecuencia, crece la probabilidad de que la red falle al recuperar los estados correctos.

2. Ejercicio 1

2.1. Consigna

Consigna: Entrene una red de Hopfield '82 con las imágenes binarias disponibles en el campus.

a) Verifique si la red aprendió las imágenes enseñadas.

b) Evalúe la evolución de la red al presentarle versiones alteradas de las imágenes aprendidas: agregado de ruido, elementos borrados o agregados.

c) Evalúe la existencia de estados espurios en la red: patrones inversos y combinaciones de un número impar de patrones. (Ver *Spurious States*, en la sección 2.2, Hertz, Krogh & Palmer, pág. 24).

d) Realice un entrenamiento con las 6 imágenes disponibles. ¿Es capaz la red de aprender todas las imágenes? Explique.

2.2. Resolución

2.2.1. Preprocesamiento

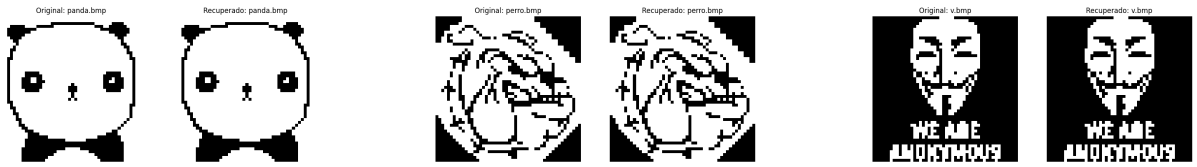
Una limitación estructural fundamental de la red de Hopfield es la imposibilidad de memorizar o procesar patrones (imágenes) de diferentes tamaños de forma simultánea. Esta restricción se debe directamente a la formulación matemática de su arquitectura:

La red está definida por una cantidad estática y fija de N neuronas. Cada imagen bidimensional que se presenta a la red debe "aplanarse" previamente hasta convertirse en un vector unidimensional de estado ξ^μ de longitud N. Consecuentemente, la matriz de pesos sinápticos w_{ij}, encargada de almacenar los patrones mediante las interconexiones, posee un tamaño estricto y predefinido de $N \times N$.

Teniendo en cuenta que el set de imágenes provisto por la catedra contenía ejemplares de distintos tamaños, el primer paso fue seleccionar un *subset* de imagenes del mismo tamaño. En este caso se seleccionaron las tres imagenes de tamaño 50×50 píxeles. Una vez aplanadas las imágenes en patrones de 2500 posiciones, ya estaban listas para entrenar a la red.

2.2.2. Verificación de entrenamiento

El siguiente paso fue verificar que la red había aprendido los patrones con los que fue entrenados correctamente. El resultado se puede observar en la figura 1. Efectivamente, aprendió los patrones ya que no modificó las imagenes que se le pasaron.



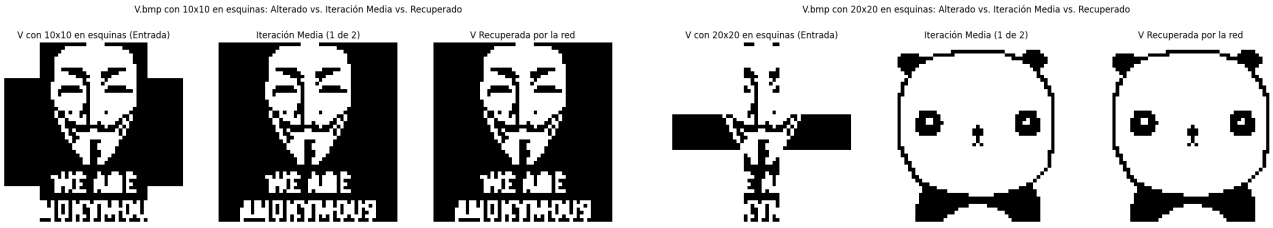
(a) Panda original vs. red. (b) Perro original vs. red. (c) Imágen 'v' original vs. red.

Figura 1: Comparación entre las imágenes originales y las que devuelve la red.

2.2.3. Patrones alterados

La próxima prueba para la red fue intentar recuperar las imágenes con las que fue entrenada, a partir de esos mismos patrones, pero alterados. La manera de probar esto fue implementar una modificación distinta a cada imágen y ver que tanto se la podía modificar antes de que la red falle en recuperar el patrón original.

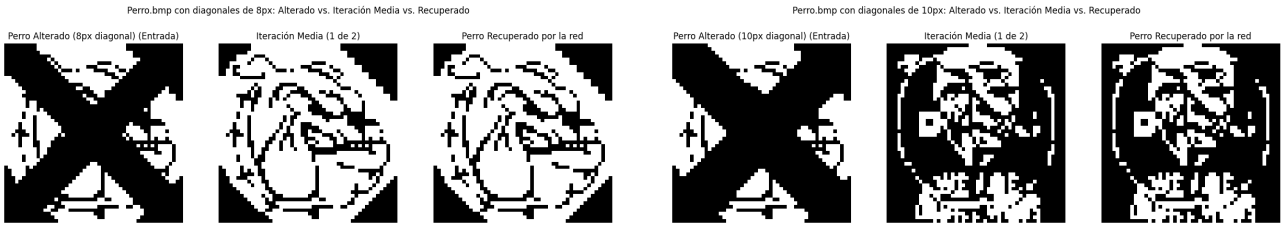
En primer lugar se probó borrando las esquinas de la imagen **v.bmp**. Los resultados se pueden observar en la figura 2. Se puede ver como en el primer caso la red puede recuperar la imagen, sin embargo en el segundo caso no. A priori este resultado tiene sentido, ya que durante el entrenamiento quedó plasmada la relación entre los píxeles del fondo de la imagen. Cuando la red ve que las esquinas están un poco blancas, pero todo lo demás está negro, oscurece el blanco acorde a los pesos entre neuronas aprendidos en el entrenamiento. En el segundo caso, al haber tanto blanco, la red vira hacia la imagen con más blanco con la que fue entrenada, el panda. También es importante notar como siempre cambia todo lo necesario en la primer iteración, mientras que la segunda solamente sirve para confirmar que no hay ningún otro cambio para hacer.



(a) Borradas con 10 píxeles de lado. (b) Borradas con 20 píxeles de lado.

Figura 2: Performance de la red ante borrado de las esquinas en diferentes medidas.

Otra prueba que se realizó fue pintar las diagonales de la imagen del perro, al igual que en el caso anterior, con diferentes grosores. Los resultados se pueden observar en la figura 3. Similarmente, el primer grosor lo soporta correctamente, devolviendo la imagen original. El segundo caso es más complicado, ya que no recupera la original pero tampoco recupera otro patrón con el que haya sido entrenado. El prestar mucha atención, se puede observar que termina en una combinación de las tres imágenes y con colores invertidos. Esto es un estado espurio, tema que se desarrollará más adelante.

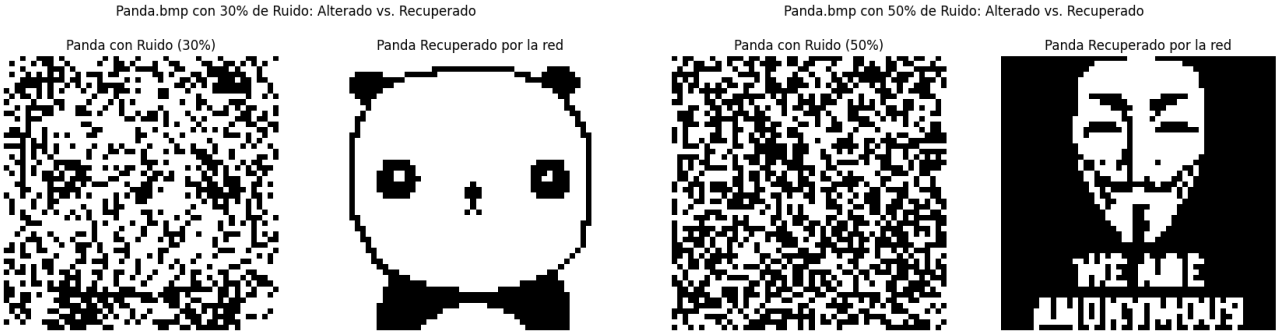


(a) Diagonales con 8 píxeles de grosor. (b) Diagonales con 10 píxeles de grosor.

Figura 3: Performance de la red ante pintado de las diagonales con distintos grosores.

Para la tercer y última imagen con la que la red fue entrenada, se introdujo distintas cantidades de ruido para ver como recuperaba las imagenes. Estos resultados se pueden ver en la figura 4. La transición se da exactamente igual que en los casos anteriores, solamente con un intermedio. De la misma manera que en las anteriores, existe un umbral donde el ruido es tanto que interpreta afinidad con otro patrón. La imagen del

panda es mayoritariamente blanca, pero con 50 % de ruido se torna bastante oscura, razón por la que la red la confunde con la imagen **v.bmp**.



(a) Ruido del 30 % en los píxeles.

(b) Ruido del 50 % en los píxeles.

Figura 4: Performance de la red ante ruido en los píxeles.

2.2.4. Estados espurios

En el ejemplo del perro con las diagonales pintadas se tuvo un primer acercamiento al fenómeno de los estados espurios. Como se menciona en la sección 1 los estados espurios pueden ser los mismos patrones invertidos o una mezcla impar de patrones. Para demostrar la existencia de los estados mencionados, se le entregó a la red la imagen del panda con los colores invertidos. El resultado fue el esperado, la red convergió a la misma imagen que le fue entregada (figura 5a). Cabe aclarar que solamente le tomó una iteración, es decir, no efectuó cambios. También pueden pasar cosas más complejas, como se obervó en la figura 3b, donde el resultado es una suerte de combinación de los tres patrones y al mismo tiempo con colores invertidos.

Los estados espurios también pueden generarse como una suma impar de estados aprendidos por la red. Para probar esto, se generó un estado espurio a partir de las tres imagenes y se lo entregó a la red para observar la salida. Teóricamente, este estado debería ser estable, por lo que la salida sería igual a la entrada. En la figura 5b se observa como efectivamente la red no modifica ningún píxel.

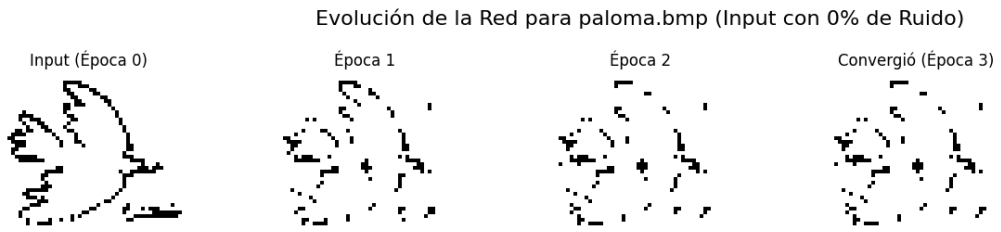
(a) Rendimiento ante un patrón invertido.

(b) Rendimiento ante un estado espurio.

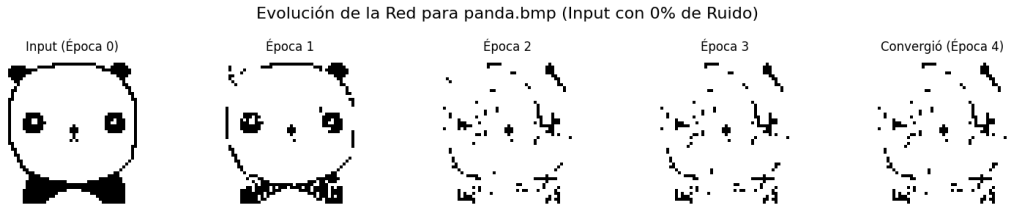
Figura 5: Análisis de los estados espurios en la red.

2.2.5. Entrenamiento de la red con los 6 patrones

Por último, adaptaron los tamaños de las 3 imagenes restantes del *dataset*, para lograr una misma cantidad de neuronas que las imagenes ya usadas. La figura 6 muestra como hubo algunos patrones que la red no pudo aprender correctamente. Esto puede ser por similaridad de colores con otra imagen, por ejemplo la paloma y el panda.



(a) Evolución de la red frente a la paloma.



(b) Evolución de la red ante el panda.

Figura 6: Rendimiento de la red con entrenamiento de 6 imágenes.

3. Ejercicio 2

3.1. Consigna

a) Comprobar estadísticamente la capacidad de la red de Hopfield '82 calculando la cantidad máxima de patrones pseudo-aleatorios aprendidos en función del tamaño de la red. Obtener experimentalmente los resultados del Cuadro 1 (los valores del cuadro corresponden a una iteración con actualización sincrónica).

$P_{\mathbf{error}}$	$p_{\mathbf{max}}/N$
0,001	0,105
0,0036	0,138
0,01	0,185
0,05	0,37
0,1	0,61

Cuadro 1: Valores teóricos de probabilidad de error y capacidad límite (Tabla 2.1, sección 2.2, Hertz, Krogh & Palmer, pág. 19).

b) Proponga una manera de generar patrones con distintos grados de correlación. Utilice el método propuesto para analizar cómo varía la capacidad de la red de Hopfield en función de la correlación entre patrones.

3.2. Resolución

3.2.1. Metodología experimental para patrones descorrelacionados

Para obtener experimentalmente los valores de capacidad requeridos en la Tabla 2.1 mencionada en la consigna, se diseñó un algoritmo que evalúa la probabilidad de error empírica de la red tras un único paso de actualización. El procedimiento computacional implementado consistió en los siguientes pasos:

1. **Generación de patrones:** Se generó un conjunto de p patrones ξ^μ de dimensión N. Para garantizar la independencia estadística requerida por el modelo, cada componente $\xi_i^\mu \in \{-1, 1\}$ fue extraída de una distribución uniforme discreta con una probabilidad de 0,5 para cada estado.

2. **Entrenamiento:** Se construyó la matriz de pesos sinápticos w_{ij} aplicando la regla de Hebb sobre los p patrones generados.

3. **Actualización sincrónica:** Se presentó cada uno de los patrones originales ξ^μ como estado inicial de la red. A continuación, se aplicó **exactamente una iteración con actualización sincrónica**, recalculando el estado de las N neuronas de manera simultánea.

4. **Cálculo de error global empírico:** Tras la actualización, se contabilizó la cantidad de bits que diferían de su estado original. La tasa de error experimental de la red (E_{total}) se calculó como la sumatoria de todos los bits erróneos dividida por la cantidad total de bits evaluados ($p \times N$).

5. **Determinación de la capacidad:** Para cada umbral de tolerancia teórica P_{error} establecido por la consigna (ej. 0,01, 0,05, etc.), se incrementó iterativamente la cantidad de patrones almacenados p. Se definió a p_{max} como la cantidad máxima de patrones que la red logró procesar cumpliendo la condición de que $E_{total} \leq P_{error}$. Finalmente, la capacidad límite para cada umbral se reportó mediante la relación p_{max}/N.

Fue entonces de esa manera que se corrieron 10 ensayos para cada valor P_{error} y así calcular la capacidad promediandolos. Los resultados se pueden observar en el Cuadro 2. Se observa como a medida que el umbral de error se hace más permisivo, la red es capaz de recordar más cantidad de patrones. Por otro lado, se puede ver la similitud con los valores teóricos del Cuadro 1. En conclusión, se puede decir que se pudo reproducir la tabla teórica exitosamente.

P_{error} **(Objetivo)**	**Capacidad Promedio** (p_{max}/N)
0.001	0.1100
0.0036	0.1400
0.01	0.1900
0.05	0.3633
0.1	0.6167

Cuadro 2: Resultados experimentales de la capacidad límite de la red en función de la probabilidad de error tolerada.

3.3. Generación de patrones correlacionados

Para evaluar el impacto de la correlación espacial en la capacidad de la red, se implementó un algoritmo basado en la generación de una familia de patrones a partir de un ancestro común. Sea $\xi^M \in \{-1, 1\}^N$ un "patrón madre" generado de forma puramente aleatoria (donde cada estado tiene probabilidad 0,5).

A partir de este patrón madre, se genera un conjunto de p patrones descendientes (*offsprings*) ξ^μ. Cada componente i de un patrón descendiente se obtiene multiplicando la componente original del patrón madre por una máscara de mutación estocástica m_i^μ:

$$\xi_i^\mu = \xi_i^M \cdot m_i^\mu \tag{6}$$

La variable aleatoria $m_i^\mu \in \{-1, 1\}$ determina si el bit se mantiene idéntico al patrón madre ($m_i^\mu = 1$) o si se invierte ($m_i^\mu = -1$). Definimos la probabilidad de inversión (*flip*) como $P(m_i^\mu = -1) = p_{flip}$. En consecuencia, el valor esperado de la máscara de mutación para cualquier componente es:

$$E[m_i^\mu] = (1) \cdot P(m_i^\mu = 1) + (-1) \cdot P(m_i^\mu = -1) = (1 - p_{flip}) - p_{flip} = 1 - 2p_{flip} \tag{7}$$

El objetivo es que cualquier par de patrones descendientes distintos, digamos ξ^μ y ξ^ν (con $\mu \neq \nu$), posean un nivel de correlación C esperado entre sí. La correlación espacial teórica se define como el valor esperado del producto de sus componentes:

$$C = E[\xi_i^\mu \xi_i^\nu] = E[(\xi_i^M m_i^\mu)(\xi_i^M m_i^\nu)] \tag{8}$$

Dado que para cualquier estado bipolar se cumple que $(\xi_i^M)^2 = 1$, la expresión se simplifica a:

$$C = E[m_i^\mu m_i^\nu] \tag{9}$$

Como las máscaras de mutación se generan de manera independiente para cada patrón descendiente, la esperanza del producto es el producto de las esperanzas:

$$C = E[m_i^\mu] \cdot E[m_i^\nu] = (1 - 2p_{flip})^2 \tag{10}$$

Para hallar la probabilidad de mutación necesaria en el algoritmo en función de la correlación deseada C, despejamos p_{flip}:

$$\sqrt{C} = |1 - 2p_{flip}| \tag{11}$$

Esta ecuación cuadrática presenta dos soluciones matemáticamente válidas para obtener la misma correlación cruzada C entre los descendientes: $p_{flip} = \frac{1-\sqrt{C}}{2}$ y $p_{flip} = \frac{1+\sqrt{C}}{2}$. En la implementación computacional de este trabajo, se optó por la segunda rama algebraica:

$$p_{flip} = \frac{\sqrt{C} + 1}{2} \tag{12}$$

De esta manera, queda demostrado que al aplicar estocásticamente esta probabilidad de inversión sobre copias de un patrón madre, los patrones resultantes exhibirán entre sí exactamente el grado de correlación C estipulado como parámetro del sistema.

3.3.1. Capacidad de la red en función de la correlación

El siguiente paso natural fue repetir el análisis de la sección 3.2.1, pero esta vez alimentando a la red con patrones con distintos grados de correlación. Se evaluó la capacidad para varios valores de correlación y de esa manera poder observar detalladamente como evoluciona esta relación. Estos resultados se pueden observar en la figura 7. Es evidente como la capacidad decae exponencialmente en todos los casos. Este resultado es consistente con la teoría, ya que a mayor correlación los patrones son cada vez más parecidos entre sí, dificultando su almancenamiento como atractores individuales.

Sin embargo, un resultado interesante es como pasando cierto valor de correlaación, algunas curvas saltan bruscamente hacia un tope de 0,66. Este tope está dado por la relación entre la cantidad de patrones máxima que el algoritmo tenía a su disposición y la cantidad de neuronas de la red, en este caso 200 y 300 respectivamente. Este salto brusco, sin embargo, no significa que la red haya mejorado su capacidad de memoria. Cuando la correlación es demasiado alta, todos los patrones que le enseñamos a la red son prácticamente idénticos entre sí. La red pierde la capacidad de distinguir las sutilezas entre un patrón y otro, y termina mezclándolos y aprendiendo un solo patrón "promedio".

Al momento de evaluar la red, le presentamos uno de estos patrones y la red nos devuelve ese único patrón promedio que se aprendió de memoria. Como el patrón original y la imagen promedio son casi exactamente iguales, la cantidad de píxeles diferentes es ínfima. El algoritmo detecta que el error es bajísimo y lo da por válido, creando la falsa sensación de que la red logró aprender y diferenciar los 200 patrones con éxito (llegando al tope de 0,66). En la realidad, la red se saturó, dejó de diferenciar imágenes y simplemente responde siempre lo mismo, pero el cálculo de error no es lo suficientemente estricto como para notarlo.

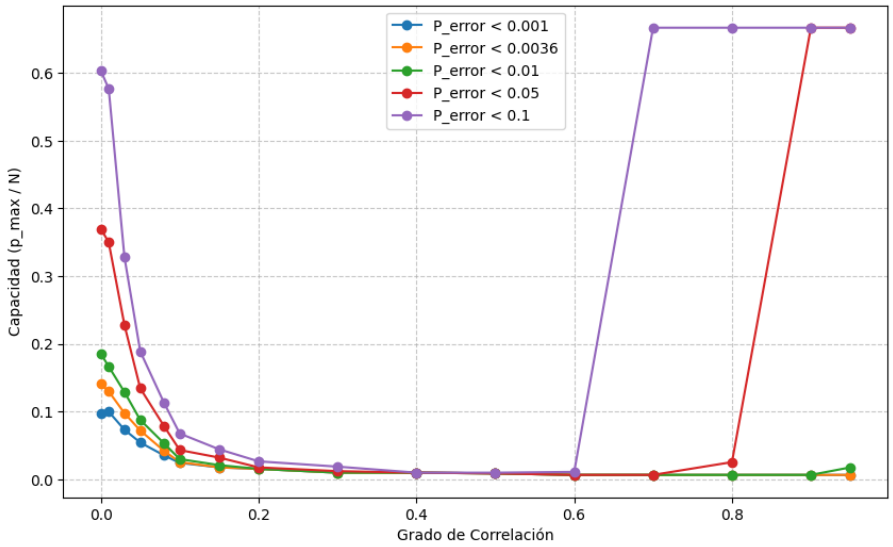


Figura 7: Relación entre correlación y capacidad de la red para distintos umbrales de error.

4. Ejercicio 3

4.1. Consigna

3. Implemente una red de Hopfield '82 que aprenda patrones pseudo-aleatorios y estudie qué sucede con los patrones aprendidos cuando algunas interconexiones son eliminadas al azar:

   a) ¿Cómo cambia el error en función del porcentaje de sinapsis eliminadas?

   b) ¿Cómo cambia la capacidad en función del porcentaje de sinapsis eliminadas?

   c) ¿Qué ocurre con la energía durante la evolución asincrónica de la red?

4.2. Resolución

4.2.1. Error frente a perdida de sinapsis

En este caso se entrenó una red de 200 neuronas con tan solo 15 patrones. Este último número se debe a que el objetivo es evaluar el error de la red atribuible al borrado de componentes de la matriz de pesos $\mathbf{W}$, no a la confusión de la red entre patrones aprendidos. Al entrenar con 15 patrones, nos encontramos por debajo del máximo teórico mencionado en la sección 1.3.

Se implementó un *loop* en el cual se fueron borrando sinapsis aleatoriamente según una lista de porcentajes fija. El error total en función del porcentaje de sinapsis borradas se puede observar en la figura 8. Se observa que la red es robusta frente a disturbios, ya que hasta aproximadamente 60 % de los pesos de la matriz borrados, el error total se mantiene cercano a 0. En el último tramo del gráfico el error total aumenta bruscamente, dibujando una curva estilo exponencial. Para el caso de todos los valores de la matriz eliminados, el error se vuelve aleatorio con probabilidad de 50 %, resultado lógico ya que se pierde la propiedad de memoria.

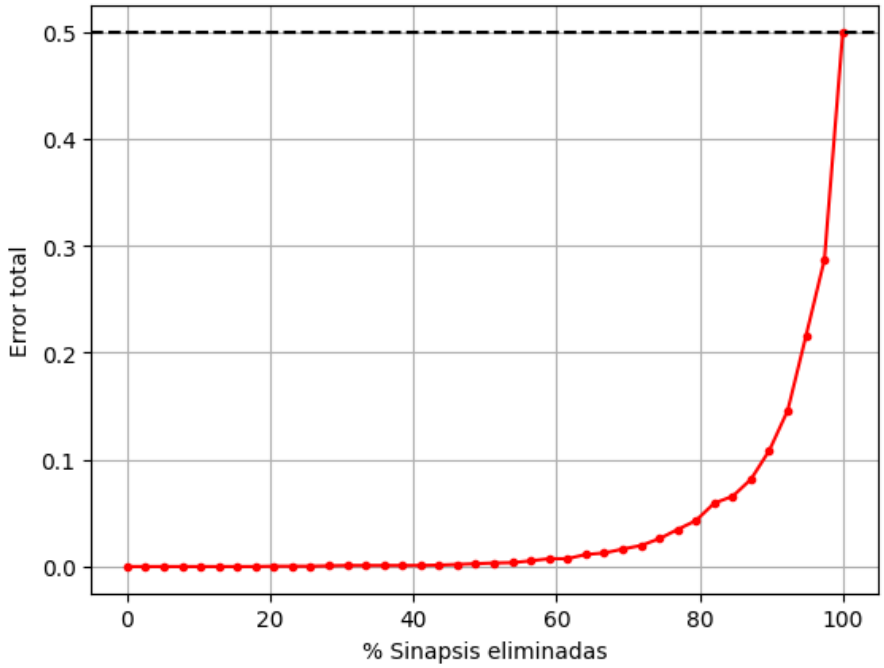


Figura 8: Error total en función del procentaje de sinapsis borradas.

4.2.2. Capacidad frente a pérdida de sinapsis

Por otro lado, se analizó también como evoluciona la capacidad de la red ante el borrado de pesos de la matriz. Se puso a disposición de la red 120 patrones, ya que es un valor cercano al máximo teórico del mayor umbral en la tabla 1. En la figura 9 se puede observar la evolución mencionada para los distintos umbrales de tolerancia usados en el ejercicio de la sección 3. El comportamiento es acorde al que se podría predecir. La capadidad en cada caso baja cuasi linealmente.

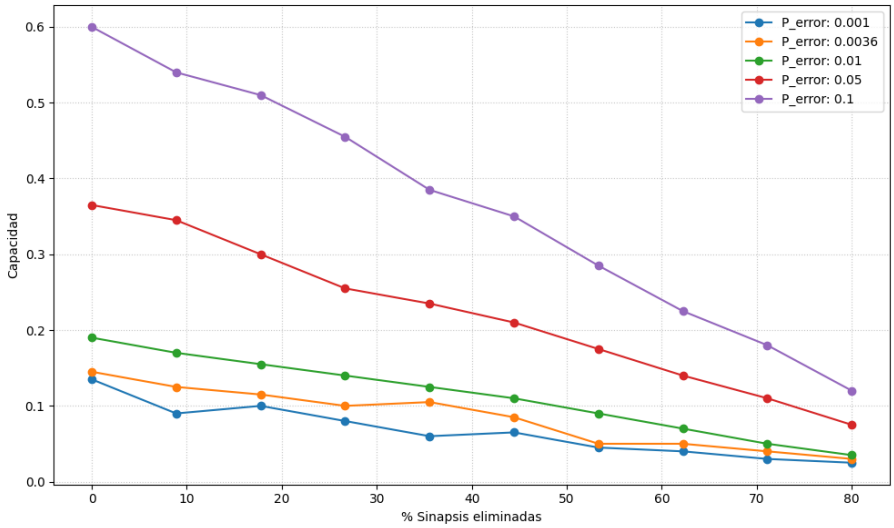


Figura 9: Capacidad en función del procentaje de sinapsis borradas.

4.2.3. Energía ante actualización asincrónica

Para este último paso, se experimentó con la función Energía.

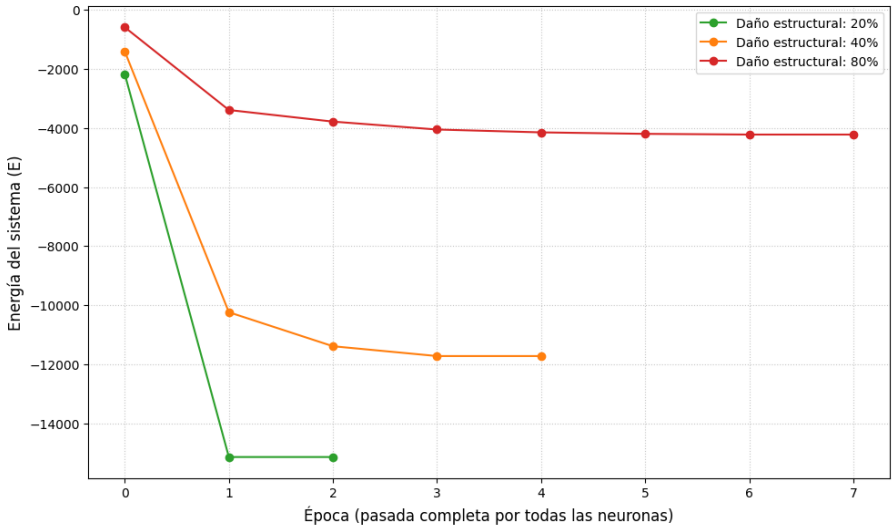


Figura 10: Energía en función de la época.

Referencias